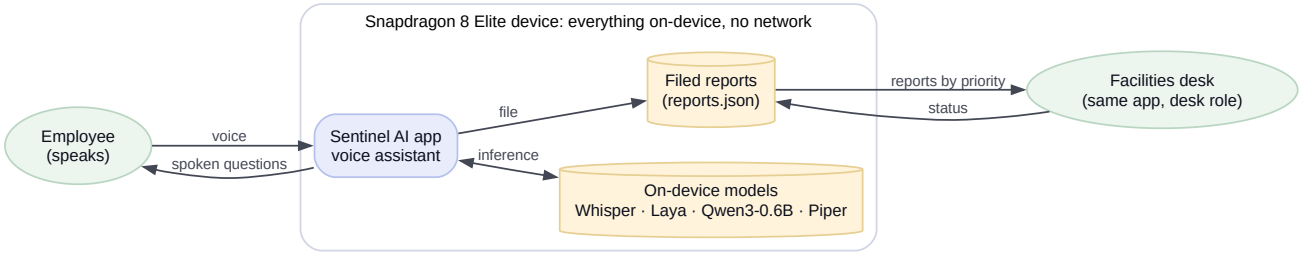
Sentinel AI: how the voice incident assistant works

Architecture, pipeline and tech stack · Snapdragon 8 Elite (SM8750) · build 7c6bb7c · 11 October 2026

One sentence. An employee taps the mic and says what's wrong; the phone transcribes it (Whisper), decides what kind of incident it is (fine-tuned Laya on the NPU), asks only for what's missing (form policy from the incident taxonomy), pulls the values out of the answers (Qwen3-0.6B with strict validation), speaks each question (Piper), and files a routed, prioritised report when the user says "done". Nothing leaves the device.

1. System context

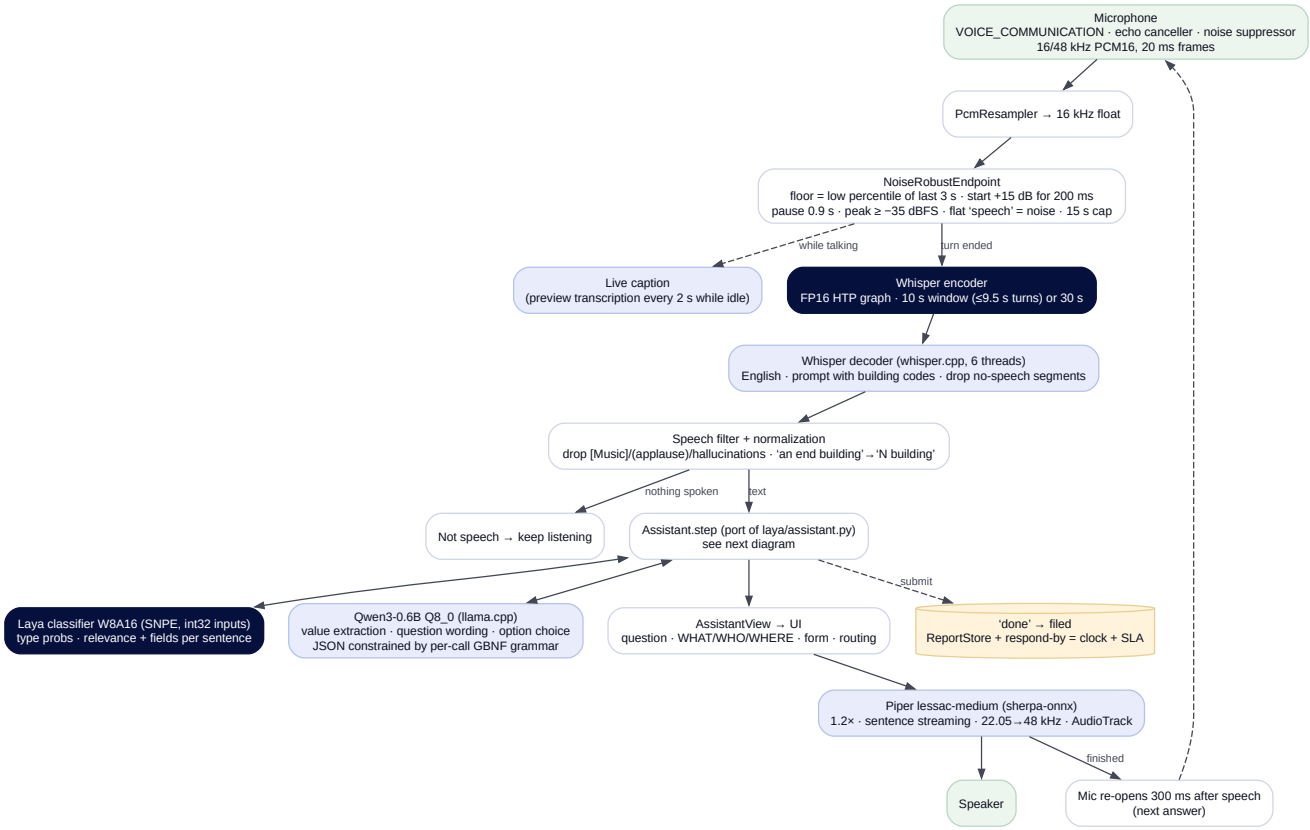


Two roles share one app on one device: a reporter who talks to the assistant, and the facilities desk that works the filed reports.

NPU (Hexagon HTP) | CPU model | app logic | user-facing / I/O | data / files

2. One conversation turn, end to end

Every user answer goes through the same path. Audio stays on the device; the heavy encoders run on the NPU, the small language model and the voice on the CPU.



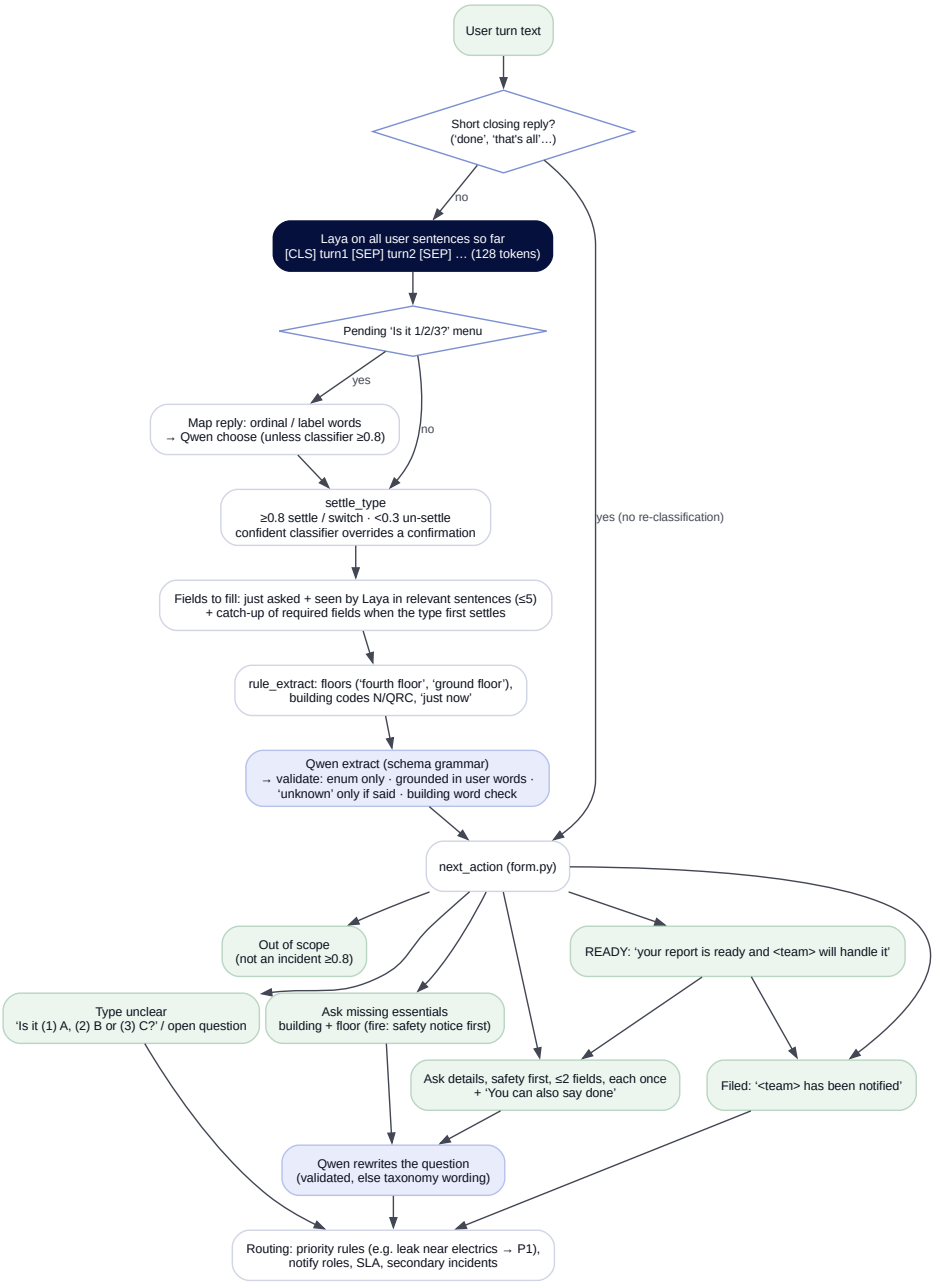
The voice path. Dashed edges are background or optional paths. Median timings on the kit: pause 0.79 s, Whisper 0.19 s, Laya 11 ms, Qwen 0.4–0.9 s per call, first reply audio 0.06 s; end of speech → reply audible 1.78 s median (see the latency report).

Stage by stage

Stage	What it does	Implementation
Capture	Opens the mic with echo cancellation and noise suppression; 20 ms PCM16 frames; never blocks on inference.	`core/audio ConversationCapture`, AudioRecord VOICE_COMMUNICATION
Resample	Any capture rate → 16 kHz mono float.	`core/inference PcmResampler`
Pause detection	Decides when the user has finished. Noise floor = 20th percentile of the last 3 s (follows fans/AC); speech starts at +15 dB for 200 ms; turn ends after 0.9 s within 6 dB of the floor; turns must peak ≥ −35 dBFS (people a few metres away are ignored); 1 s of flat loud sound is learned as new background; 15 s cap.	`core/inference NoiseRobustEndpoint` (7 unit tests)
Live caption	While the user talks, the audio so far is transcribed every 2 s if the engine is idle, for the on-screen caption.	`IncidentEngine` previews
Speech recognition	Whisper base. Encoder on the NPU through whisper.cpp's external-encoder hook: 10 s window graph for turns ≤ 9.5 s (18 ms), 30 s graph otherwise (190 ms). Decoder on the CPU. Language fixed to English; initial prompt lists the building codes; segments with no-speech probability ≥ 0.6 dropped.	`core/native-whisper` (whisper.cpp d1be6fde + `whisper_snpe_encoder.cpp`)

Speech filter	Removes non-speech tags ([Music], (applause), ♪) and Whisper's noise hallucinations ("Thank you."); if nothing is left the mic just keeps listening. Normalises domain mishearings: "an end / M building" → "N building", "Q.R.C." → "QRC", lone "Dunn" → "done".	`IncidentEngine.spoken / normalizeSpeech` (tested)
Assistant step	Classify → settle type → extract values → choose next action → word the question → route. Exact port of `laya/assistant.py`, `form.py`, `llm.py`.	`app/…/assistant/` (8 parity tests vs Python)
Voice reply	Piper lessac-medium at 1.2×, synthesised sentence by sentence while the previous sentence plays; resampled to 48 kHz so it uses the device's primary audio mixer.	`engine/NeuralSpeaker`, sherpa-onnx 1.13.8
Turn-taking	When a question has been spoken, the mic re-opens 300 ms later by itself. Tapping during speech interrupts and listens immediately.	`IncidentAssistantViewModel`
Filing	"done" (or the button) files the report; respond-by time = filing time from the device clock + the priority's SLA.	`data/ReportStore` (atomic JSON)

3. Inside the assistant step



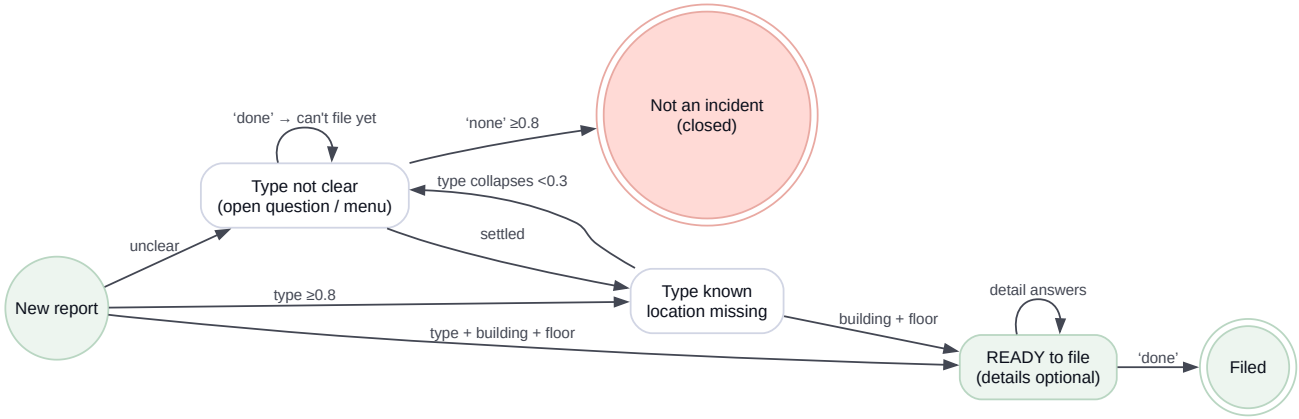
Decision flow per turn (port of laya/assistant.py + form.py). The language model never decides the incident type or the priority; those come from Laya and from the taxonomy rules.

Rules the code guarantees

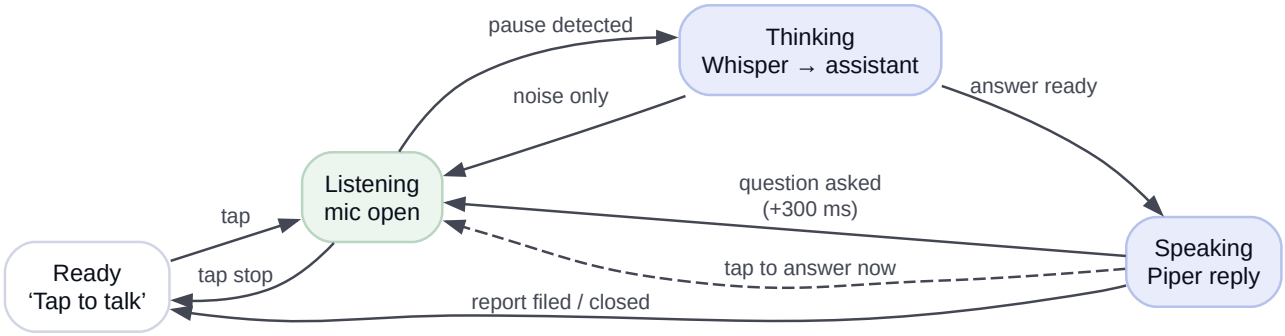
- **Essentials first.** A report needs WHAT (incident type, which also decides WHO: the department) and WHERE (building and floor). Until they are known only those are asked; "done" early gets "I can't file this yet...".
- **Type settling.** Settle at ≥ 0.8 (calibrated with temperature 1.5733); switch when another type reaches 0.8; un-settle if the settled type drops below 0.3; below 0.8 offer the top-3 menu ("Is it (1) A, (2) B or (3) C?") if they cover ≥ 50 %, else ask an open question; a confident classifier beats the user's menu answer.
- **Details after READY.** Safety fields first, then location, then type-specific, then time; at most two fields per question, each asked once, each ending "(You can also say 'done' to file the report now.)".
- **Values must be grounded.** Enums only from the allowed list; text/number values must appear in the user's words; "unknown" only if the user said they don't know; a building must be a known code or contain a building word. Deterministic rules run first (floors incl. "fourth/ground floor", building codes, "just now").

- **Routing from the taxonomy.** Default priority per type, raised by rules (e.g. water near electrics → P1 + electrician + EHS; an unknown safety field counts as the worse case); extra roles notified; secondary incidents flagged when the transcript mentions them; SLA from the priority.
- **Out of scope.** If Laya is ≥ 0.8 sure it is not an incident, the assistant says so and files nothing.

4. Conversation and screen states

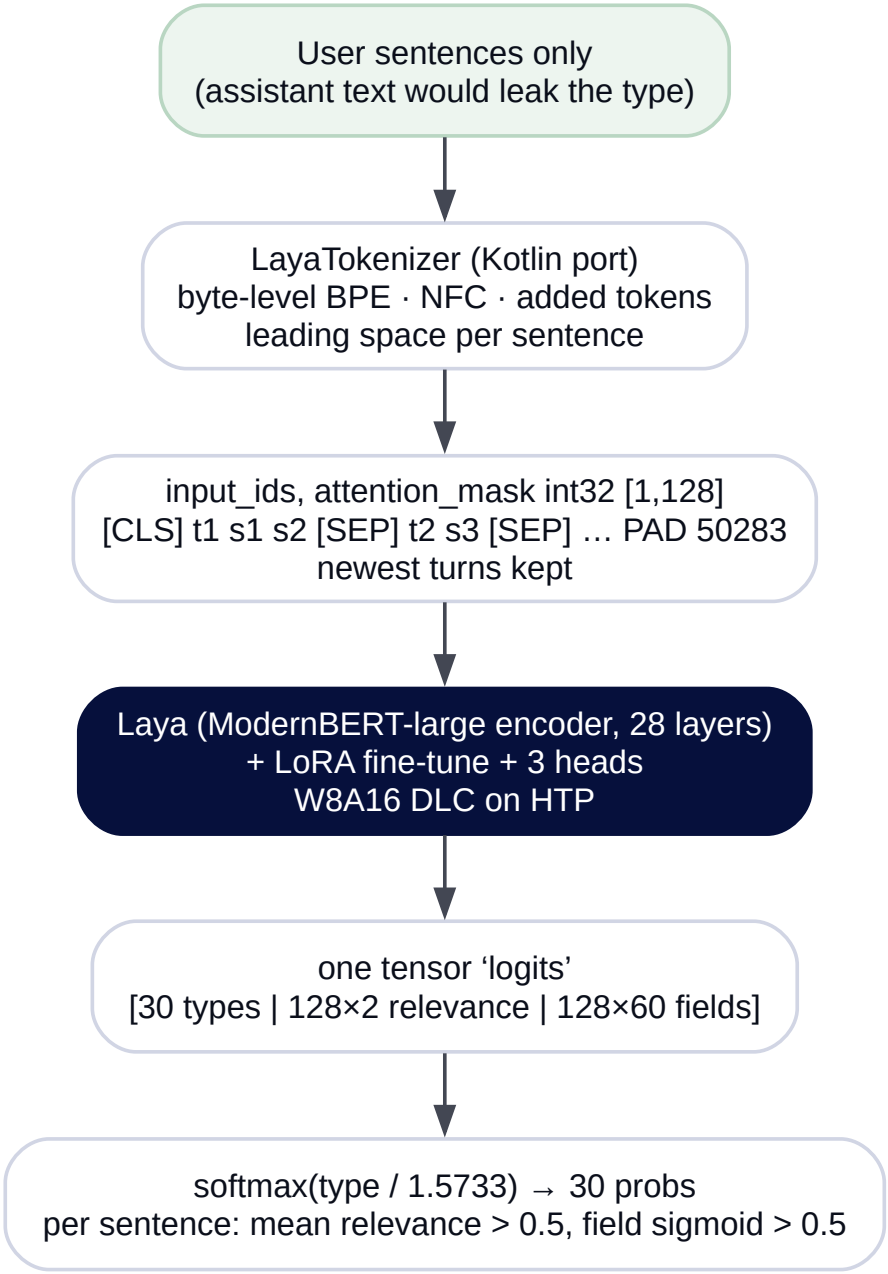


Report states. READY means the essentials are known and the user can file at any time.



What the voice screen shows. The mic opens by itself after every question.

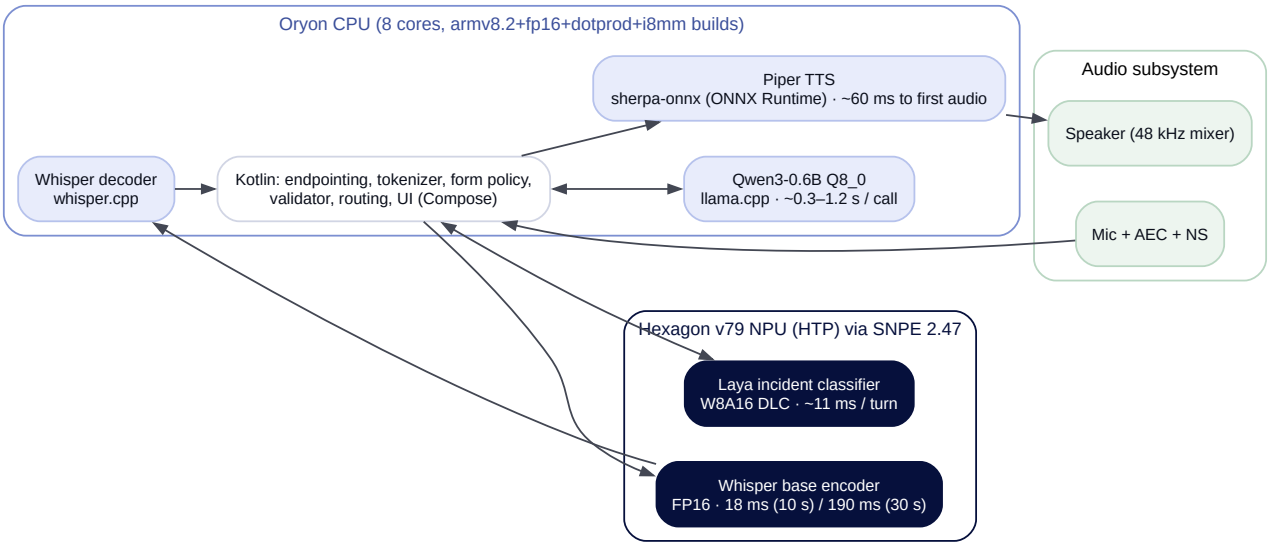
5. The incident classifier (Laya)



Classifier input and output. One NPU inference per turn (~11 ms) returns the incident type and, per sentence, whether it is relevant and which form fields it mentions.

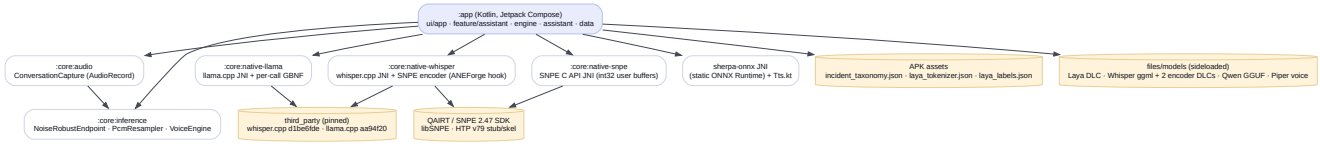
Laya is convaiinnovations' Laya encoder (ModernBERT-large, 28 layers) fine-tuned with LoRA on a synthetic conversation dataset built from the incident taxonomy (1,292 conversations, 28 types plus "not an incident" and multi-incident cases). It reads only the user's sentences, because the assistant's questions would give away the type. It reaches 87.6 % type accuracy on the test set running W8A16 on the NPU (88.0 % in fp32; the same model without fine-tuning scores 11.6 %). Exporting it for the NPU needed two fixes: attention masks bounded at −60 instead of float-min (which collapses the quantized attention range), and one concatenated output tensor (SNPE on the HTP only returns the last output of a multi-output graph). The Kotlin tokenizer reproduces the Hugging Face tokenizer token for token (tested), and the input layout matches the Python `build_samples` exactly.

6. What runs where



Hardware mapping on the Snapdragon 8 Elite. Native code is compiled for the Oryon cores' FP16, dot-product and int8-matmul instructions; without that Whisper took minutes per call.

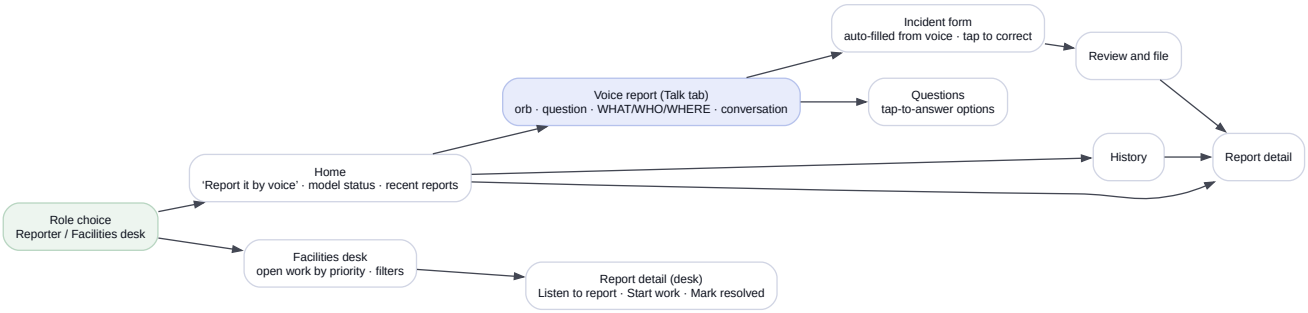
7. Code structure



Gradle modules and what they depend on. Model files are not in the APK; they are copied to the app's private storage.

Package / path	Contents
app/.../assistant	`Form.kt` (taxonomy, FormState, settle_type, next_action, routing) · `Extract.kt` (prompts, GBNF grammars, validator, rule_extract) · `Assistant.kt` (turn loop) · `Classifier.kt` (Laya input/output) · `LayaTokenizer.kt`
app/.../engine	`IncidentEngine` (audio → Whisper → assistant, model loading, speech filter) · `NativeModels` (JNI bindings, WhisperSpeech) · `SnpeLayaRunner` · `LlamaLlm` · `NeuralSpeaker` / `SystemSpeech` · main's original engines (unused by the UI)
app/.../feature/assistant	`IncidentAssistantViewModel`: capture, engine, speech, turn-taking, filing, desk read-aloud
app/.../ui/app	Compose screens: role, home, voice report, form, questions, review, history, report detail, desk; components per the design system
app/.../data	`ReportStore`, `FiledReport`, `ReportStatus`
app/src/debug	`ProbeActivity` (adb-only via DUMP permission): scripted turns, Whisper/TTS timing, room levels, `VoiceBench` latency benchmark
core/*	audio capture; endpointing and resampling; JNI to whisper.cpp, llama.cpp and SNPE
scripts/	`prepare_native.sh` · `push_models_kit.sh` · `fetch_sherpa_onnx.sh` · `whisper_npu/` · `probe_kit.sh` · `latency_report.py` · `docs/`

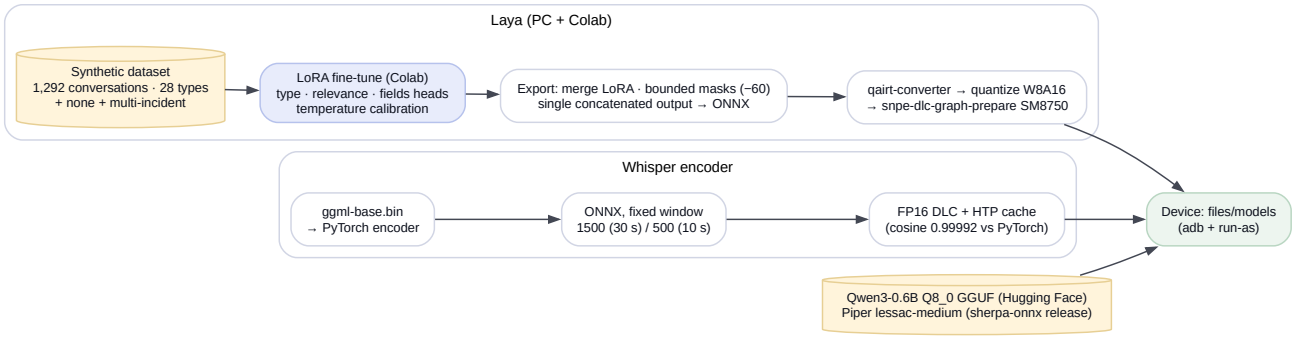
8. Screens



Navigation. The reporter side is voice only; the form and question screens are views onto the same conversation.

Design system: navy `#06103C` primary, cool slate surfaces, crimson only for live recording, P1 and missing essentials; Roboto Flex editorial display + UI text type scale; Aurora voice orb (respects reduced motion); 48 dp touch targets; labeled top-bar actions.

9. How the models are built and deployed



Offline pipeline. Laya and the Whisper encoder are compiled for the SM8750 HTP ahead of time; Qwen and Piper are used as published.

Model	File on device	Size	Runs on
Laya (fine-tuned)	`laya_w8a16_htp.dlc`	860 MB	NPU, SNPE
Whisper base (multilingual)	`ggml-base.bin` + `whisper_base_encoder_fp16_htp.dlc` (30 s) + `…_10s_fp16_htp.dlc`	148 + 262 + 90 MB	encoder NPU, decoder CPU
Qwen3-0.6B	`Qwen3-0.6B-Q8_0.gguf`	639 MB	CPU, llama.cpp
Piper en_US lessac-medium	`voice/` (ONNX + espeak-ng data)	79 MB	CPU, sherpa-onnx

Models are copied into the app's private storage with `scripts/push_models_kit.sh` (Android 15 blocks apps from reading files adb creates in their external folder). The APK carries the taxonomy, the Laya tokenizer and the label list. All models load once at app start (about 1.4 s) and stay resident (about 2.5 GB).

10. Tech stack

Layer	Technology
Device	Qualcomm Snapdragon 8 Elite dev kit (SM8750P), Hexagon v79 NPU (HTP), 8 Oryon cores, 15 GB RAM, Android 15
App	Kotlin 1.9.22, Jetpack Compose (BOM 2024.02, Material 3), Navigation Compose, AndroidX ViewModel/StateFlow; minSdk 24, target 34
Build	Gradle 8.11.1, AGP 8.5, JDK 17, NDK (27.2 default, 30 on Linux), CMake; native builds tuned with `GGML_CPU_ARM_ARCH=armv8.2-a+fp16+dotprod+i8mm`
NPU runtime	Qualcomm QAIRT / SNPE 2.47 (C API, int32 and float user buffers, HTP v79 stub/skel, FastRPC)
Speech recognition	whisper.cpp (pinned), Whisper base; encoder exported PyTorch → ONNX → FP16 DLC
Language model	llama.cpp (pinned), Qwen3-0.6B Q8_0, thinking off, JSON forced by per-call GBNF grammars
Classifier	Laya (ModernBERT-large) + LoRA (PyTorch, Colab) → ONNX → W8A16 DLC; Kotlin byte-level BPE tokenizer
Voice	sherpa-onnx 1.13.8 (ONNX Runtime statically linked), Piper VITS en_US lessac-medium, AudioTrack 48 kHz
Audio	AudioRecord VOICE_COMMUNICATION, AcousticEchoCanceler, NoiseSuppressor, custom energy endpointing
Data	`incident_taxonomy.json` (28 types, 6 departments, 50+13 fields, P1–P4 SLAs 15/60/240/1440 min, rules); reports as JSON in private storage
Tooling	Python 3 (PyTorch, tokenizers, onnx) for export and parity fixtures; Graphviz, matplotlib, headless Chromium for docs; adb/run-as for deployment

11. Quality, privacy and performance

Tests. 8 parity tests against the Python reference (tokenizer, classifier input, rules, validator, end phrases, six scripted conversations with every action, value and routing, selftest cases, grammar); 7 endpoint tests on synthetic noise; 3 speech-filter/normalization tests; core pipeline tests from main. On-device checks with the probe.

Privacy. No network use; reports stay in app-private storage; transcripts are never written to the system log; the debug probe can only be started from adb (DUMP permission) and is absent from release builds.

Performance (kit, 23 voice turns). End of speech → reply audible: 1.78 s median, 2.79 s p90. After the pause is detected: 1.0 s. First turn of a report 2.73 s (catch-up extraction), follow-ups 1.64 s. 19/23 turns transcribed word-perfect. CPU 0.67 cores median; no thermal pressure.

Known limits. English only; single-letter building names are fragile (mitigated); a nearby person's clear sentence can still be taken as a turn; the stock 0.6B model misses some values (fine-tune planned); all models resident need about 2.5 GB.